Project Title	**Rising Waters: Machine Learning Based Flood Prediction System**		
Developer	M. Thrishna Tanmayee	**Team ID**	[Team ID]
Date	26 June	**Phase / Document**	Requirement Analysis

6. Solution Requirements

Requirements were split into functional requirements (what the system must do) and non-functional requirements (the qualities the system must have while doing it), following the standard split used in the design-thinking phases above. Each functional requirement below traces directly to a concrete piece of code in app.py or train.py, so that the requirement list can be verified against the implementation rather than treated as an aspirational document.

6.1 Functional Requirements

ID	Requirement
FR-1	Accept user-provided meteorological inputs (temperature, humidity, cloud cover, seasonal and annual rainfall).
FR-2	Provide an Auto-Fill option to instantly populate the form with realistic demo values.
FR-3	Scale input features using the same Standard Scaler used during training.
FR-4	Predict flood probability and classify it into Normal / Elevated / Severe tiers.
FR-5	Generate dynamic, human-readable insights explaining the prediction.
FR-6	Display recommended safety protocols corresponding to the severity tier.
FR-7	Handle invalid/missing input gracefully and surface an error message in the UI.

6.2 Non-Functional Requirements

ID	Requirement
NFR-1	Usability — responsive, single-page interface usable on desktop and mobile.
NFR-2	Performance — prediction response returned within a few seconds of form submission.
NFR-3	Maintainability — training (train.py) and serving (app.py) logic kept separate and independently runnable.
NFR-4	Portability — trained artifacts serialized via Joblib for reuse without retraining.
NFR-5	Reliability — model and scaler existence validated at startup before serving requests.

NFR-5 in particular reflects a deliberate defensive choice in app.py: the application raises a File Not Found Error at import time if flood_model.joblib or scaler. joblib are missing, rather than failing silently or crashing on the first prediction request. This trades a slightly less friendly startup error for a guarantee that, if the server is running at all, it is running with a valid model.